

TECHNICAL DESIGN & DEVELOPER GUIDE

# Pandemic Simulation System

Agent-Based SEIR-V Epidemic Modelling Engine  
with Interactive GUI and Real-Time Visualisation

Document Type  
Technical Design Document

Audience  
Software Developers / Engineers

Status  
Active — Version 1.0

**Prerequisites:** Readers are expected to have working knowledge of object-oriented programming (OOP), Python variables and data types, and familiarity with third-party library usage. Prior exposure to NumPy and Matplotlib is beneficial but not mandatory.

# Table of Contents

---

- 1 Introduction ..... 4
  - 1.1 Purpose and Scope ..... 4
  - 1.2 System Goals ..... 4
  - 1.3 Technology Stack ..... 4
- 2 System Architecture ..... 5
  - 2.1 High-Level Overview ..... 5
  - 2.2 Module Responsibilities ..... 6
- 3 Data Flow ..... 6
  - 3.1 End-to-End Pipeline ..... 6
  - 3.2 Parameter Dictionary Schema ..... 7
- 4 Class and Function Breakdown ..... 8
  - 4.1 Class: `App` — `main_ui.py` ..... 8
  - 4.2 Function: `run_pandemic_simulation(params)` — `simulation.py` ..... 9
- 5 Core Simulation Logic ..... 10
  - 5.1 Compartmental Model: SEIR-V ..... 10
  - 5.2 State Transition Diagram ..... 11
  - 5.3 Movement Engine ..... 11
  - 5.4 Infection Engine ..... 12
  - 5.5 Variant Mutation Logic ..... 12
  - 5.6 Hospital Capacity Effect ..... 13
- 6 Configuration and Parameters ..... 13
  - 6.1 Parameter Sourcing ..... 13
  - 6.2 Recommended Parameter Ranges ..... 13
- 7 How to Modify and Extend ..... 14
  - 7.1 Adding a New Disease State ..... 14
  - 7.2 Modifying the Transmission Model ..... 15
  - 7.3 Performance Optimisation ..... 15
  - 7.4 Adding Intervention Features ..... 16
- 8 Best Practices and Architectural Notes ..... 16
  - 8.1 Current Architecture Assessment ..... 16
  - 8.2 Proposed OOP Refactoring ..... 17
  - 8.3 Code Quality Guidelines ..... 18
- 9 Future Development Roadmap ..... 18
  - 9.1 Short-Term Improvements (v1.1) ..... 18
  - 9.2 Medium-Term Improvements (v2.0) ..... 18
  - 9.3 Long-Term Vision (v3.0) ..... 19
- 10 Appendix: Quick Reference ..... 19
  - 10.1 State Integer Codes ..... 19

10.2 Complete Simulation Loop Pseudocode ..... 19

# 1 Introduction

## 1.1 Purpose and Scope

The **Pandemic Simulation System** is a desktop application designed to model and visualise the spread of an infectious disease through a population of agents over a configurable number of simulation days. The system combines an interactive graphical user interface (GUI) with a mathematically grounded simulation engine, enabling researchers, students, and developers to explore epidemic dynamics under a wide variety of conditions.

This document serves as both a **Technical Design Document** and a **Developer Guide**. It describes the system’s architecture, the rationale behind design decisions, the internal data-flow, and practical instructions for extending or modifying the codebase. It is intended for developers who are comfortable with Python, object-oriented concepts, and numerical computing libraries.

## 1.2 System Goals

The primary objectives of the system are as follows:

- **Scientific accuracy:** Implement a well-established compartmental epidemic model (SEIR-V) augmented with agent-based spatial dynamics.
- **Interactivity:** Allow users to configure all epidemiological and demographic parameters through a clean GUI before each simulation run.
- **Observability:** Render a live, real-time animated visualisation of agent states alongside time-series plots of compartment populations.
- **Extensibility:** Structure the codebase so that new disease states, transmission mechanisms, or intervention strategies can be added with minimal friction.
- **Reporting:** Automatically export simulation results to a structured Excel spreadsheet for downstream analysis.

## 1.3 Technology Stack

The system is implemented entirely in Python and depends on the following libraries:

| Library       | Module | Role in System                                                        |
|---------------|--------|-----------------------------------------------------------------------|
| customtkinter | ctk    | Modern-themed Tkinter widgets for the GUI layer                       |
| numpy         | np     | Vectorised numerical computation — positions, distances, random draws |
| matplotlib    | plt    | Real-time animated scatter plots and time-series graphs               |

scipy

spatial

Efficient KD-tree spatial queries for proximity-based infection

openpyxl

–

Writing simulation output to .xlsx report files

## 2 System Architecture

### 2.1 High-Level Overview

The system is divided into two clearly separated layers: the **Presentation Layer** (`main_ui.py`) and the **Simulation Engine Layer** (`simulation.py`). Communication between layers is unidirectional — the UI collects user parameters and passes them as a plain Python dictionary to the engine, which owns all simulation state and produces all outputs.



## 2.2 Module Responsibilities

### 2.2.1 `main_ui.py` — Presentation Layer

This module is responsible for every aspect of user interaction. It constructs the application window, lays out labelled input fields for each configurable parameter, performs input validation (type checking, range enforcement), and finally invokes the simulation engine when the user clicks **Run Simulation**.

The module contains a single class, `App`, which inherits from `ctk.CTk` (a subclass of `tkinter.Tk`). All UI state is encapsulated within this class.

### 2.2.2 `simulation.py` — Engine Layer

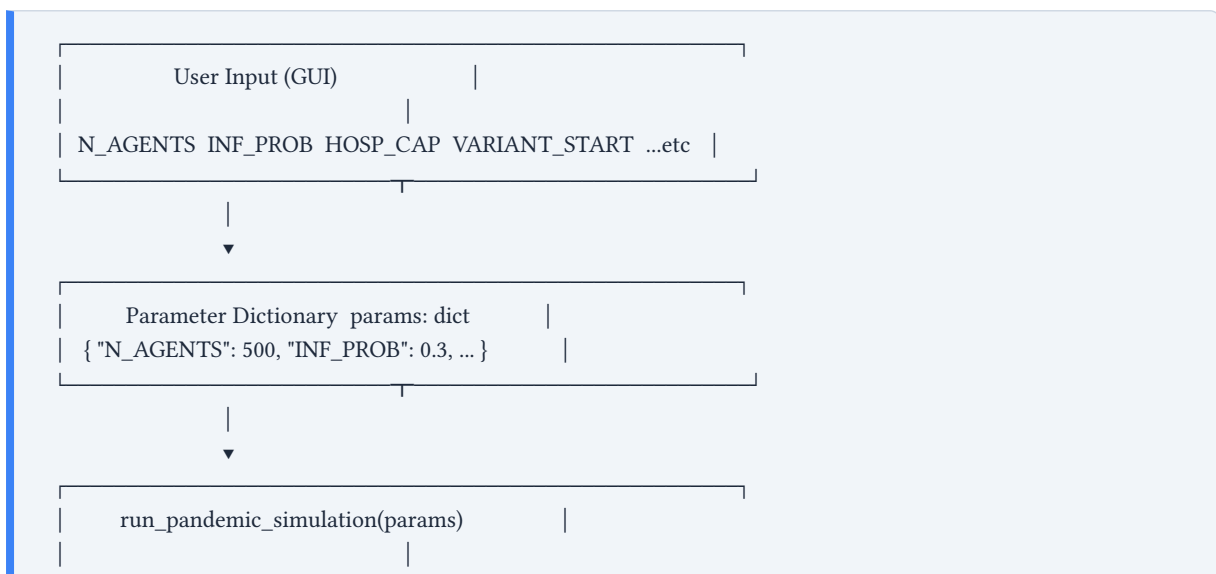
This module contains the entire simulation logic. The entry point is the top-level function `run_pandemic_simulation(params)`, which accepts the parameter dictionary produced by the UI. Internally, this function orchestrates agent initialisation, the day-by-day simulation loop, and all rendering and export logic.

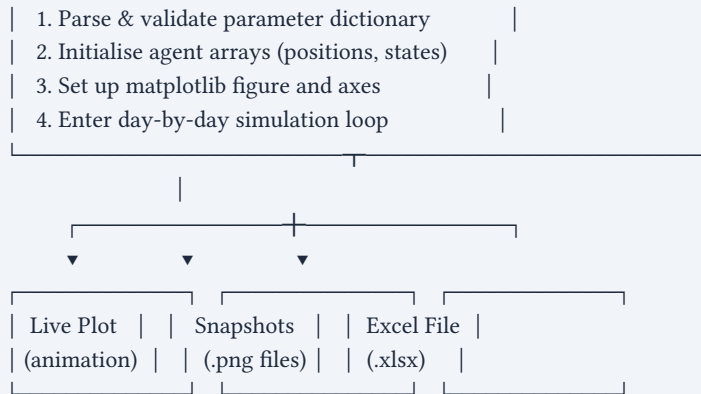
🔴 **Note:** In the current implementation, `simulation.py` is **procedural** rather than object-oriented. A refactoring roadmap toward a fully object-oriented architecture is discussed in Section 9.

## 3 Data Flow

### 3.1 End-to-End Pipeline

The following diagram illustrates the complete data flow from user input to simulation output:





## 3.2 Parameter Dictionary Schema

The `params` dictionary acts as the sole interface between the UI and the engine. Every key maps to a scalar numeric value. The following table documents all currently supported keys:

| Key                        | Type  | Range      | Description                                                               |
|----------------------------|-------|------------|---------------------------------------------------------------------------|
| <code>N_AGENTS</code>      | int   | 50–5000    | Total number of agents in the simulation                                  |
| <code>INF_PROB</code>      | float | 0.0–1.0    | Probability of transmission per contact event                             |
| <code>INF_RADIUS</code>    | float | 1–50       | Spatial radius (pixels) within which infection can occur                  |
| <code>INCUBATION</code>    | int   | 1–30       | Days an agent spends in the Exposed state before becoming Infectious      |
| <code>INF_DURATION</code>  | int   | 1–60       | Days an agent remains in the Infected state before recovery or death      |
| <code>DEATH_PROB</code>    | float | 0.0–1.0    | Per-day probability of death for an Infected agent                        |
| <code>HOSP_CAP</code>      | int   | 0–N_AGENTS | Hospital capacity; exceeding it increases the effective death probability |
| <code>VAX_RATE</code>      | float | 0.0–1.0    | Daily fraction of Susceptible agents that receive vaccination             |
| <code>VARIANT_START</code> | int   | 0–DAYS     | Simulation day on which a new, more transmissible variant emerges         |
| <code>VARIANT_MULT</code>  | float | 1.0–10.0   | Multiplier applied to <code>INF_PROB</code> when the variant is active    |
| <code>DAYS</code>          | int   | 1–1000     | Total number of days to simulate                                          |
| <code>SPEED</code>         | float | 0.1–10.0   | Agent movement speed in pixels per day                                    |

## 4 Class and Function Breakdown

### 4.1 Class: `App` — `main_ui.py`

`App` inherits from `customtkinter.CTk` and serves as the root window of the application. It is instantiated once at program startup and runs the Tkinter event loop via `app.mainloop()`.

#### 4.1.1 Constructor: `__init__(self)`

The constructor performs three distinct tasks:

1. **Window configuration:** Sets title, geometry, and colour theme.
2. **Widget construction:** Iterates over a parameter specification list to create labelled `CTkEntry` widgets for each configurable value, populating them with sensible default values.
3. **Button construction:** Creates a **Run Simulation** `CTkButton` bound to `self.start_sim`.

```
class App(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.title("Pandemic Simulation")
        self.geometry("480x720")

        # --- Build input widgets dynamically ---
        self.inputs = {}
        param_specs = [
            ("N_AGENTS", "Number of Agents", "500"),
            ("INF_PROB", "Infection Probability", "0.30"),
            ("INF_RADIUS", "Infection Radius (px)", "15"),
            ("INCUBATION", "Incubation Period (days)", "5"),
            ("INF_DURATION", "Infectious Duration (days)", "10"),
            ("DEATH_PROB", "Death Probability", "0.02"),
            ("HOSP_CAP", "Hospital Capacity", "50"),
            ("VAX_RATE", "Daily Vaccination Rate", "0.01"),
            ("VARIANT_START", "Variant Emergence Day", "30"),
            ("VARIANT_MULT", "Variant Transmissibility ×", "1.5"),
            ("DAYS", "Simulation Duration (days)", "100"),
            ("SPEED", "Agent Speed", "1.0"),
        ]
        for key, label, default in param_specs:
            # ... create CTkLabel + CTkEntry ...
            self.inputs[key] = entry_widget

        ctk.CTkButton(self, text="▶ Run Simulation",
                      command=self.start_sim).pack(pady=20)
```



#### 4.1.2 Method: `start_sim(self)`

This method is the bridge between the UI and the engine. Its execution follows a strict three-phase pattern:

1. **Validation:** Each entry value is tested with `float()` inside a `try/except` block. Non-numeric or out-of-range values produce an error dialog rather than a crash.
2. **Conversion:** All values are cast to their required Python types and assembled into the `params` dictionary.
3. **Dispatch:** `run_pandemic_simulation(params)` is called, transferring control entirely to the engine layer.

```
def start_sim(self):
    params = {}
    try:
        for key, entry in self.inputs.items():
            raw = entry.get().strip()
            params[key] = float(raw) # all parsed as float first
        # Post-process integer fields
        for int_key in ("N_AGENTS", "INCUBATION", "INF_DURATION",
                       "HOSP_CAP", "VARIANT_START", "DAYS"):
            params[int_key] = int(params[int_key])
    except ValueError as exc:
        messagebox.showerror("Invalid Input", str(exc))
    return

run_pandemic_simulation(params)
```

#### 4.2 Function: `run_pandemic_simulation(params)` — `simulation.py`

This is the single most important function in the codebase. It owns all simulation state and drives every phase of execution. Its internal structure is outlined below:

```
run_pandemic_simulation(params)
|
|—— Phase 1 — Initialisation
| |—— Unpack scalar parameters from dict
| |—— Allocate numpy arrays:
| | |—— positions : float32[N, 2] — (x, y) coordinates
| | |—— velocities : float32[N, 2] — (vx, vy) direction vectors
| | |—— states : int8[N] — compartment label (S/E/I/R/D/V)
| | |—— timers : int16[N] — days spent in current state
| |—— Seed initial Infected agents (typically 3–5)
|
|—— Phase 2 — Figure Setup
| |—— Create matplotlib Figure with 2 subplots
| | |—— Subplot A: scatter plot of agent positions (colour = state)
| | |—— Subplot B: time-series line plot of compartment counts
| |—— Enable interactive / animation mode
```

```

|
|   └── Phase 3 – Simulation Loop (for day in range(DAYS))
|   |   └── [3a] Variant Mutation
|   |   |   └── if day >= VARIANT_START → current_inf_prob *= VARIANT_MULT
|   |   |
|   |   └── [3b] Movement Engine
|   |   |   └── positions += velocities * SPEED
|   |   |   └── Boundary reflection (elastic collision with walls)
|   |   |
|   |   └── [3c] Infection Engine
|   |   |   └── Build KD-tree from positions of Infected agents
|   |   |   └── Query all Susceptible agents within INF_RADIUS
|   |   |   └── Apply Bernoulli trial with current_inf_prob → Exposed
|   |   |
|   |   └── [3d] Vaccination Engine
|   |   |   └── Randomly select VAX_RATE × |S| agents → Vaccinated
|   |   |
|   |   └── [3e] State Transition Engine
|   |   |   └── E agents: timer++ ; if timer ≥ INCUBATION → Infected
|   |   |   └── I agents: timer++ ; if timer ≥ INF_DURATION →
|   |   |       └── death roll < DEATH_PROB (×hospital_factor) → Dead
|   |   |       └── otherwise → Recovered
|   |   |
|   |   └── [3f] Logging
|   |   |   └── Append daily counts (S, E, I, R, D, V) to history arrays
|   |   |
|   |   └── [3g] Rendering
|   |   |   └── Update scatter plot colours and positions
|   |   |   └── Update time-series lines
|   |   |   └── Call plt.pause(0.01) to flush to screen
|   |
|   └── Phase 4 – Export
|       └── Write daily history arrays to Excel (.xlsx) via openpyxl

```

## 5 Core Simulation Logic

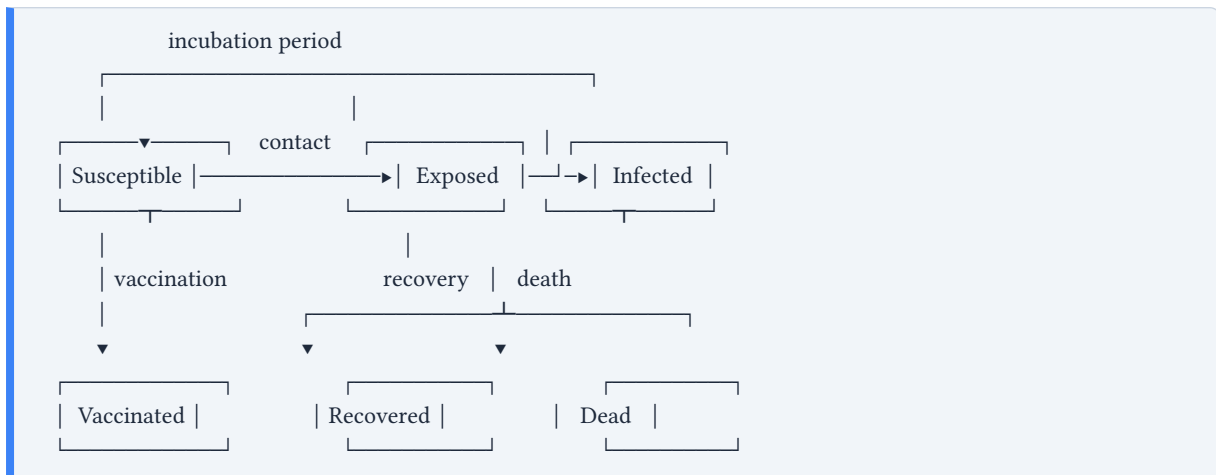
### 5.1 Compartmental Model: SEIR-V

The simulation implements an **agent-based** realisation of the classical SEIR model, extended with **Dead (D)** and **Vaccinated (V)** compartments. Unlike equation-based SEIR models that track population fractions, this system assigns a discrete state to every individual agent, enabling spatial and stochastic heterogeneity.

The six compartments and their integer codes are:

| Code     | Integer  | Colour | Meaning                                                  |
|----------|----------|--------|----------------------------------------------------------|
| <b>S</b> | <b>0</b> | Blue   | Susceptible — healthy, can become exposed                |
| <b>E</b> | <b>1</b> | Orange | Exposed — infected but not yet infectious; incubating    |
| <b>I</b> | <b>2</b> | Red    | Infected — actively infectious; can transmit to S agents |
| <b>R</b> | <b>3</b> | Green  | Recovered — immune; cannot be reinfected in this model   |
| <b>D</b> | <b>4</b> | Black  | Dead — removed from infection dynamics                   |
| <b>V</b> | <b>5</b> | Purple | Vaccinated — immune via vaccination; cannot be infected  |

## 5.2 State Transition Diagram



Formally, the per-day transition probabilities are:

- $S \rightarrow E$ : occurs when agent  $i$  (Susceptible) is within **INF\_RADIUS** of at least one Infected agent, and a Bernoulli draw with probability  $p_{\text{inf}}$  succeeds.
- $E \rightarrow I$ : occurs deterministically when **timer[i] >= INCUBATION**.
- $I \rightarrow R$ : occurs when **timer[i] >= INF\_DURATION** and the death roll fails.
- $I \rightarrow D$ : occurs when **timer[i] >= INF\_DURATION** and **random() < p\_"death" × h\_"factor"**, where  $h_{\text{factor}} \geq 1$  when hospital capacity is exceeded.
- $S \rightarrow V$ : occurs for a random fraction **VAX\_RATE** of Susceptible agents per day.

## 5.3 Movement Engine

Each agent maintains a two-dimensional position vector  $\mathbf{r}_i \in [0, W] \times [0, H]$  and a velocity vector  $\mathbf{v}_i$  with unit magnitude scaled by **SPEED**. Each day, positions are updated as:

$$\mathbf{r}_i \leftarrow \mathbf{r}_i + \mathbf{v}_i \times \text{SPEED}$$

When an agent reaches a boundary, the perpendicular component of its velocity is negated (elastic reflection), producing a billiard-ball-like bouncing motion:

```
# Vectorised boundary reflection
out_x = (positions[:, 0] < 0) | (positions[:, 0] > WIDTH)
out_y = (positions[:, 1] < 0) | (positions[:, 1] > HEIGHT)
velocities[out_x, 0] *= -1
velocities[out_y, 1] *= -1
positions = np.clip(positions, [0, 0], [WIDTH, HEIGHT])
```

## 5.4 Infection Engine

The infection calculation is the computationally heaviest step, as it involves pairwise proximity checks. A naïve  $O(N^2)$  double loop would be prohibitively slow for large populations; the system therefore uses a **KD-tree** (via `scipy.spatial.KDTree`) to reduce the complexity to  $O(N \log N)$ :

```
from scipy.spatial import KDTree

infected_positions = positions[states == I]
susceptible_mask = states == S

if len(infected_positions) > 0:
    tree = KDTree(infected_positions)
    # Find all susceptible agents within INF_RADIUS of any infected agent
    neighbours = tree.query_ball_point(
        positions[susceptible_mask], r=INF_RADIUS
    )
    # Stochastic transmission
    for idx, contacts in enumerate(neighbours):
        if len(contacts) > 0:
            if np.random.rand() < current_inf_prob:
                states[susceptible_indices[idx]] = E
                timers[susceptible_indices[idx]] = 0
```

## 5.5 Variant Mutation Logic

On `VARIANT_START` day, the effective infection probability is scaled upward:

```
if day == params["VARIANT_START"]:
    current_inf_prob = min(
        params["INF_PROB"] * params["VARIANT_MULT"],
        1.0 # probability is clamped to [0, 1]
    )
```

This models the emergence of a more transmissible pathogen variant mid-simulation, enabling study of second-wave dynamics.

## 5.6 Hospital Capacity Effect

When the number of Infected agents exceeds `HOSP_CAP`, the effective death probability is amplified:

```
n_infected = np.sum(states == I)
if n_infected > HOSP_CAP:
    hospital_factor = 1.0 + (n_infected - HOSP_CAP) / HOSP_CAP
else:
    hospital_factor = 1.0

effective_death_prob = DEATH_PROB * hospital_factor
```

This models the increased case-fatality rate observed when healthcare systems are overwhelmed.

## 6 Configuration and Parameters

### 6.1 Parameter Sourcing

All parameters originate exclusively from the GUI. There are no hard-coded magic numbers inside `simulation.py`; every tuneable value is read from the `params` dictionary. This design ensures that:

- The engine is fully stateless and deterministic given the same `params` and random seed.
- Automated testing can invoke the engine without launching the GUI.
- Parameters can trivially be loaded from a JSON or YAML file if batch-mode execution is needed in the future.

### 6.2 Recommended Parameter Ranges

**⚠ Warning:** Setting `INF_PROB` > 0.8 combined with `INF_RADIUS` > 30 and `N_AGENTS` > 1000 will cause an almost instantaneous epidemic that peaks within 2–3 days. This is epidemiologically unrealistic and stresses the renderer. Use these values only for stress-testing.

The table below summarises empirically validated parameter ranges that produce realistic and visually informative simulations:

| Parameter               | Conservative | Moderate | Aggressive |
|-------------------------|--------------|----------|------------|
| <code>N_AGENTS</code>   | 200          | 500      | 1500       |
| <code>INF_PROB</code>   | 0.10         | 0.30     | 0.70       |
| <code>INF_RADIUS</code> | 8            | 15       | 30         |

|              |       |      |       |
|--------------|-------|------|-------|
| INCUBATION   | 7     | 5    | 2     |
| INF_DURATION | 14    | 10   | 5     |
| DEATH_PROB   | 0.005 | 0.02 | 0.10  |
| VAX_RATE     | 0.02  | 0.01 | 0.005 |
| DAYS         | 200   | 100  | 60    |

## 7 How to Modify and Extend

### 7.1 Adding a New Disease State

As an example, suppose you wish to add a **Quarantine** state **Q** for agents who are identified and isolated.

**Step 1** — Define the integer constant:

```
S, E, I, R, D, V, Q = 0, 1, 2, 3, 4, 5, 6
```

**Step 2** — Register a colour in the visualisation colour map:

```
COLOR_MAP = {
    S: "steelblue",
    E: "orange",
    I: "red",
    R: "green",
    D: "black",
    V: "purple",
    Q: "gold",      # ← new state colour
}
```

**Step 3** — Add a UI input for the quarantine probability parameter (e.g., **QUAR\_PROB**) in **main\_ui.py**.

**Step 4** — Implement the transition logic in the state-transition block:

```
# Inside the state-transition loop
if states[i] == I and timer[i] >= DETECTION_DELAY:
    if np.random.rand() < params["QUAR_PROB"]:
        states[i] = Q  # isolate the agent
        # Quarantined agents do not move — handled in movement engine
```

**Step 5** — Exclude **Q** agents from the movement engine:

```
mobile_mask = ~np.isin(states, [D, Q]) # Dead and Quarantined don't move
positions[mobile_mask] += velocities[mobile_mask] * SPEED
```

**Step 6** — Add the **Q** count to the logging and reporting arrays.

✓ **Tip:** Keep state integer codes contiguous and document them in a module-level constant block. This prevents bugs when iterating over states numerically.

## 7.2 Modifying the Transmission Model

The infection probability draw in the current implementation is a simple Bernoulli trial. You can enrich it by incorporating agent-level attributes such as age, immune status, or mask compliance:

```
# Example: age-stratified infection risk
def transmission_prob(base_prob, age_factor, mask_factor):
    """Compute effective transmission probability for a contact."""
    return min(base_prob * age_factor * (1.0 - mask_factor), 1.0)

# Usage inside infection engine:
for idx, contacts in enumerate(neighbours):
    if len(contacts) > 0:
        p = transmission_prob(
            current_inf_prob,
            age_factors[susceptible_indices[idx]],
            mask_compliance[susceptible_indices[idx]],
        )
        if np.random.rand() < p:
            states[susceptible_indices[idx]] = E
```

To support this, you would need to initialise **age\_factors** and **mask\_compliance** arrays at simulation startup, either as constants or as random draws from empirically informed distributions.

## 7.3 Performance Optimisation

For population sizes above 2,000 agents, the Python-level loops over agent arrays can become a bottleneck. The recommended approach is **full NumPy vectorisation**:

```
# — Vectorised State Timer Increment —————
# Instead of:
for i in range(N_AGENTS):
    if states[i] in (E, I):
        timers[i] += 1

# Use:
timers[(states == E) | (states == I)] += 1

# — Vectorised E → I Transition —————
```

```
# Instead of:
for i in range(N_AGENTS):
    if states[i] == E and timers[i] >= INCUBATION:
        states[i] = I; timers[i] = 0

# Use:
advance = (states == E) & (timers >= INCUBATION)
states[advance] = I
timers[advance] = 0
```

This can achieve  $10\times$ – $50\times$  speedups depending on population size and hardware.

## 7.4 Adding Intervention Features

The following interventions are architecturally straightforward to implement:

| Intervention      | Affected Engine  | Implementation Approach                                                             |
|-------------------|------------------|-------------------------------------------------------------------------------------|
| Mask Wearing      | Infection        | Multiply <code>INF_PROB</code> by <code>(1 - mask_eff)</code> for masked agents     |
| Social Distancing | Movement         | Reduce <code>SPEED</code> by a factor; add repulsion force between agents           |
| Lockdown          | Movement         | Set <code>SPEED = 0</code> for all non-essential agents on trigger day              |
| Contact Tracing   | State Transition | Expose contacts of detected Infected agents with probability <code>TRACE_EFF</code> |
| Booster Doses     | Vaccination      | Re-vaccinate <code>R</code> agents after <code>IMMUNITY_WANING</code> days          |

## 8 Best Practices and Architectural Notes

### 8.1 Current Architecture Assessment

The current codebase is **functional** — it works correctly and produces scientifically plausible results. However, as a single large procedure, `run_pandemic_simulation` accumulates responsibilities that ideally belong to distinct, independently testable components. The following issues should be addressed in a future refactoring sprint:

- **God function anti-pattern:** `run_pandemic_simulation` mixes business logic, rendering, and I/O. This makes unit testing extremely difficult.



- **Tight coupling:** The simulation loop directly calls `plt.pause()`, binding the engine to a specific rendering backend.
- **No separation of concerns:** Movement, infection, and state-transition logic are interleaved within the same loop body.

## 8.2 Proposed OOP Refactoring

A clean object-oriented redesign would decompose the system into the following classes:

```
class Agent:
    """Represents a single individual in the simulation."""
    state : int      # compartment (S/E/I/R/D/V)
    position : np.ndarray # [x, y]
    velocity : np.ndarray # [vx, vy]
    timer : int      # days in current state
    age : float      # age in years (for stratified mortality)

class MovementEngine:
    """Handles spatial update of agent positions."""
    def step(self, agents: list[Agent], speed: float,
            bounds: tuple) -> None: ...

class InfectionEngine:
    """Handles transmission events using KD-tree proximity search."""
    def step(self, agents: list[Agent], inf_prob: float,
            inf_radius: float) -> None: ...

class StateTransitionEngine:
    """Handles compartment transitions and timer management."""
    def step(self, agents: list[Agent], params: dict) -> None: ...

class Simulation:
    """Orchestrates all engines across the simulation timeline."""
    def __init__(self, params: dict): ...
    def run(self) -> SimulationResult: ...

class SimulationResult:
    """Immutable record of daily compartment counts."""
    history : dict[str, list[int]]
    def to_excel(self, path: str) -> None: ...
```

This architecture enables:

- **Independent unit testing** of each engine with mock agent lists.
- **Swappable backends:** the `Simulation` class can accept a renderer interface, allowing matplotlib, a web canvas, or a headless mode.

- **Parallelism:** multiple `Simulation` instances with different `params` can run concurrently for parameter sweeps.

✓ **Tip:** If you undertake this refactoring, migrate incrementally — extract one engine at a time and validate against the original outputs before proceeding to the next.

## 8.3 Code Quality Guidelines

The following conventions should be observed throughout the codebase:

1. **Type annotations:** All function signatures should include PEP 484 type hints. The `params` dictionary should be typed as `dict[str, float | int]`.
2. **Docstrings:** Every public function and class should have a NumPy-style docstring documenting parameters, return values, and any raised exceptions.
3. **Constants:** All magic numbers (compartment codes, default colours, boundary sizes) should be module-level constants written in `UPPER_SNAKE_CASE`.
4. **Random seeding:** Accept an optional `seed: int | None = None` parameter and call `np.random.seed(seed)` at the start of `run_pandemic_simulation` to enable reproducible runs.
5. **Logging:** Replace bare `print()` calls with the standard `logging` module. Use `logging.DEBUG` for per-day metrics and `logging.INFO` for phase-level events.

## 9 Future Development Roadmap

### 9.1 Short-Term Improvements (v1.1)

- Extract movement, infection, and state-transition logic into separate module-level functions with full type annotations and docstrings.
- Add a `--headless` command-line flag that disables rendering and runs the simulation at maximum speed for batch parameter sweeps.
- Implement a random-seed input field in the GUI to allow reproducible runs.
- Replace the flat `params` dictionary with a typed `dataclass` or `pydantic` model to enforce constraints at construction time.

### 9.2 Medium-Term Improvements (v2.0)

- Full OOP refactoring as described in Section 8.
- Add a **network-based** transmission model as an alternative to spatial proximity: agents are nodes in a social graph, and infection spreads along edges.
- Implement **age-stratified** mortality and hospitalisation rates using publicly available IFR tables.

- Add an  $R_0$  (basic reproduction number) estimator that computes and displays the effective reproductive number  $R_t$  in real time.

### 9.3 Long-Term Vision (v3.0)

- Port the engine to a compiled backend (Numba JIT or Cython) for population sizes of 100,000+.
- Expose a REST API so that the simulation engine can be driven programmatically from Jupyter notebooks or external scripts.
- Implement a web-based visualisation frontend using D3.js or Plotly Dash, eliminating the dependency on a local display.

## 10 Appendix: Quick Reference

### 10.1 State Integer Codes

```
S = 0 # Susceptible
E = 1 # Exposed
I = 2 # Infected
R = 3 # Recovered
D = 4 # Dead
V = 5 # Vaccinated
```

### 10.2 Complete Simulation Loop Pseudocode

```
FUNCTION run_pandemic_simulation(params):

    # — Initialisation —————
    unpack all scalar values from params
    positions ← uniform random (N × 2) within [0, W] × [0, H]
    velocities ← random unit vectors scaled by SPEED
    states ← S for all agents
    timers ← 0 for all agents
    seed 3–5 agents as I (Infected)

    history ← {S:[], E:[], I:[], R:[], D:[], V:[]}

    # — Day Loop —————
    FOR day IN range(DAYS):

        IF day == VARIANT_START:
            current_inf_prob ← min(INF_PROB × VARIANT_MULT, 1.0)
```

```
# Movement
positions[mobile] += velocities[mobile] × SPEED
reflect velocities at boundaries

# Infection (KD-tree)
tree ← KDTree(positions[states == I])
FOR each S agent within INF_RADIUS of tree:
  IF rand() < current_inf_prob:
    state ← E; timer ← 0

# Vaccination
randomly select floor(VAX_RATE × |S|) S agents → V

# State Transitions
timers[(states==E) | (states==I)] += 1
states[(states==E) & (timers>=INCUBATION)] ← I; reset timers
FOR each I agent with timer >= INF_DURATION:
  IF rand() < DEATH_PROB × hospital_factor:
    state ← D
  ELSE:
    state ← R

# Hospital Capacity
hospital_factor ← 1 + max(0, (|I| - HOSP_CAP) / HOSP_CAP)

# Logging
FOR compartment IN (S, E, I, R, D, V):
  history[compartment].append(count(states == compartment))

# Render
update scatter plot colours
update time-series lines
plt.pause(0.01)

# — Export —————
write history to Excel file
```

### End of Document

For questions, contributions, or bug reports, please open an issue in the project repository.  
All proposed changes to the simulation engine should be accompanied by a test case  
that validates compartment counts against a known analytical result.